

giselleeliasi Document database design and security details 

772b30f · 5 days ago



181 lines (139 loc) · 16.5 KB

 main [CMSI4071-Group12 / documents / Software-Design-Description.md](#) Top

Preview

Code

Blame



Raw



6.1 Introduction

This document presents the architecture and detailed design for the fika web application. The project is a dedicated platform for discovering, reviewing, and logging visits to coffee shops based on specific criteria like parking, seating, and amenity availability. The application condenses scattered coffee shop information from various platforms (e.g., Google Reviews, TikTok) into a single, specialized tool.

6.1.1 System Objectives

The primary objective of the fika application is to provide a single, specialized web-based tool for coffee shop enthusiasts to discover, review, and log their visits, focusing on details often missing from general review platforms.

The key goals and objectives are:

- **Discovery and Filtering:** To provide a simple interface for searching and filtering cafes based on criteria pertinent to coffee shop goers, such as parking availability, seating capacity, Wi-Fi, and outlet availability.
- **Comprehensive Cafe Information:** To display individual cafe pages with user-provided reviews, aggregated ratings, and crucial metadata like hours, address, and amenities.
- **User Engagement:** To enable authenticated users to submit new reviews and

maintain a personalized log/favorites list of the cafes they have visited.

- **Data Visualization:** To display charts visualizing trends in cafe data to help users make informed choices.
- **Performance and Scalability:** To maintain a smooth and responsive experience, ensuring search results are returned within 3 seconds, and the system can support at least 5,000 concurrent users with 99% monthly uptime.

6.1.2 Hardware Interfaces

The fika application interfaces with specific hardware, software, and human elements in both its execution environment and its use of third-party APIs.

6.1.2.1 Hardware Interfaces

- **Client Hardware:** The system relies on standard client-side hardware interfaces, including the Network (Broadband internet connection is required for access) , Display (minimum 1280x800 resolution), Mouse/Trackpad, and Keyboard for user input and navigation.
- **Deployment Hardware:** The application is hosted on cloud infrastructure. The backend/database uses a Cloud-hosted Processor (minimum 2 vCPUs) and RAM (4 GB minimum, scalable with usage). It requires Public internet access with HTTPS/TLS for network communication.

6.1.2.2 Software Interfaces

The system integrates with several third-party software components and APIs:

- **Next.js (latest LTS):** Used for the Frontend development framework, providing server-side rendering and a modern React development environment.
- **PostgreSQL 15+:** The primary database, managed via Supabase, stores cafe records, user accounts, and reviews. It is essential for storing cafe metadata using advanced indexing and JSONB support.
- **Supabase:** Serves as the hosting platform for the backend and managed PostgreSQL database. It is chosen for its ease of integration with PostgreSQL and real-time APIs.
- **Vercel:** Used for hosting the Next.js frontend, ensuring automatic CI/CD integration for frontend code changes.
- **Supabase Authentication:** Handles all user login, signup, and account management, securing access to logging features.
- **OpenStreetMaps API:** Used to provide geographical data on cafes and to visualize

cafe locations on the Discover Page map. It is a free and open alternative to commercial mapping APIs.

6.1.2.3 Human Interfaces

- **User Interface (UI):** The system provides a web-based Graphical User Interface (GUI) accessible via a standard web browser (Chrome, Safari, etc.). The interface is divided into key areas:
 - **Discover Page:** Provides the interface to search, filter, and view cafes on an interactive map.
 - **Cafe Page:** Displays individual cafe details, reviews, and data trends.
 - **User Logging (Favorites/Reviews):** Allows users to save cafes and input new reviews.
 - **Authentication UI:** Provides dedicated login/register screens.

6.2 Architectural Design

The fika application is based on a three-tier architecture consisting of the Presentation Layer (Frontend), the Application/Business Layer (Backend API and Logic), and the Data Layer (Database). This structure ensures separation of concerns, scalability, and maintainability.

The system is partitioned into the following four top-level Computer Software Configuration Items (CSCI):

1. **Frontend CSC:** The user-facing web application.
2. **Backend CSC:** The core business logic, data management, and API endpoints.
3. **Authentication CSC:** The dedicated system for user account management, managed by Supabase.
4. **Hosting & Infrastructure CSC:** The deployment and environment configuration.

6.2.1 Major Software Components

The major software components are derived from the CSCI breakdown:

- **Frontend CSC (Presentation Layer):** Built with Next.js. It contains the user interface components:
 - **Discover Page CSU:** Manages cafe search, filtering, and map visualization (using OpenStreetMaps).

- **Cafe Page CSU:** Renders individual cafe details, aggregated ratings, and trend visualizations (using Vega charts).
- **User Logging CSU:** Manages user-specific actions like saving favorite cafes and providing an interface for writing new reviews.
- **Backend CSC (Application/Data Layer):** Hosted on Supabase and powered by PostgreSQL
 - **Database CSU:** Manages the database schema for storing CafeTable (details, attributes), Review Table (user reviews, ratings), and UserTable (authentication-linked user data).
- **Authentication CSC:** A dedicated Supabase-based system for user login, signup, and account management
 - **Auth CSU:** Manages the core login, logout, and secure session management.
 - **AuthUI module:** Provides the user interface for login and registration screens.
- **Hosting & Infrastructure CSC:**
 - **Hosting CSU:** Manages the CI/CD pipelines and deployment of the application components. This includes VercelDeploy (frontend) and SupabaseDeploy (backend services).

6.2.2 Major Software Interactions

Communication in the fika architecture is primarily through a client-server model over HTTPS/TLS.

- **Frontend-Backend/API Interaction:**
 - The Frontend CSC (Next.js) communicates with the Backend CSC's API CSU via REST/GraphQL endpoints.
 - The Backend API calls abstract the direct connection to the PostgreSQL database (Database CSU), ensuring business logic is enforced before data retrieval or persistence.
- **Authentication Interaction:**
 - The Frontend CSC interfaces with the Supabase Authentication CSC for user sign-up and login.
 - The Supabase system provides a secure authentication token upon successful

login, which the Frontend uses in subsequent API calls to the Backend.

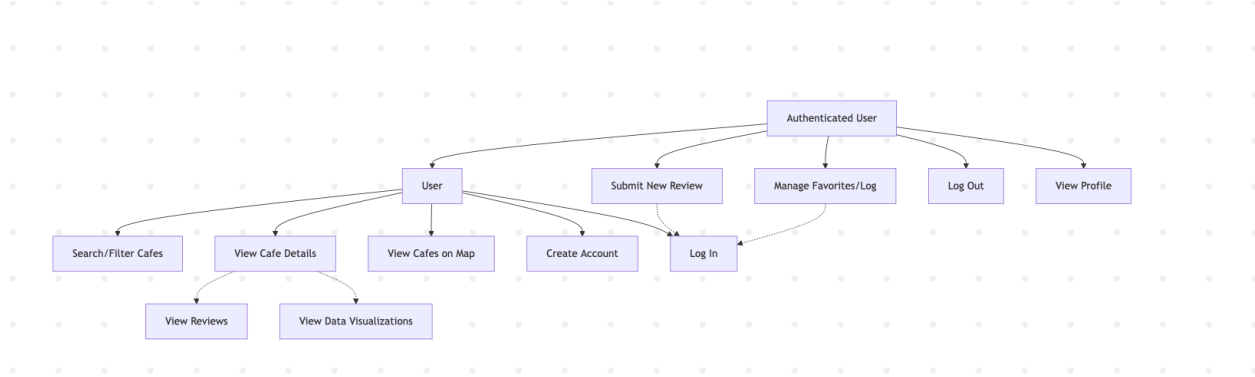
- The Backend uses the authentication token to verify the user's identity and restrict access to features like submitting reviews or accessing saved cafes.

- **External API Interaction:**

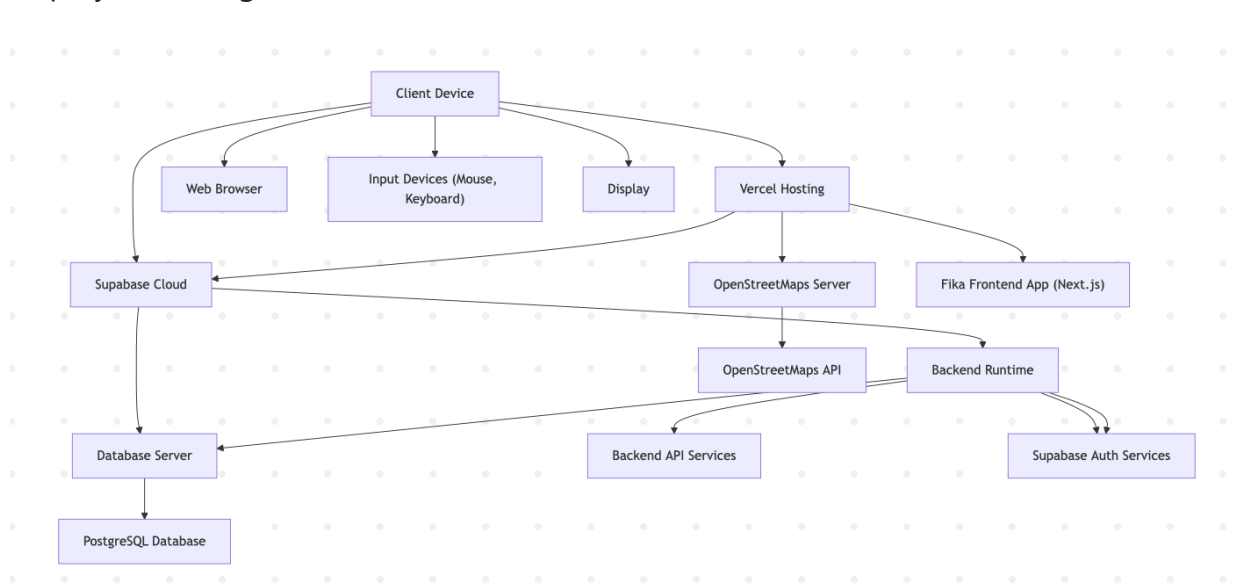
- The Frontend's MapView module uses the OpenStreetMaps API to render geographical data and cafe locations.
- The Frontend's Cafe Page CSU uses the Vega/Vega-Lite library to render visualizations after retrieving the necessary aggregated data from the Backend API

6.2.3 Architectural Design Diagrams Section

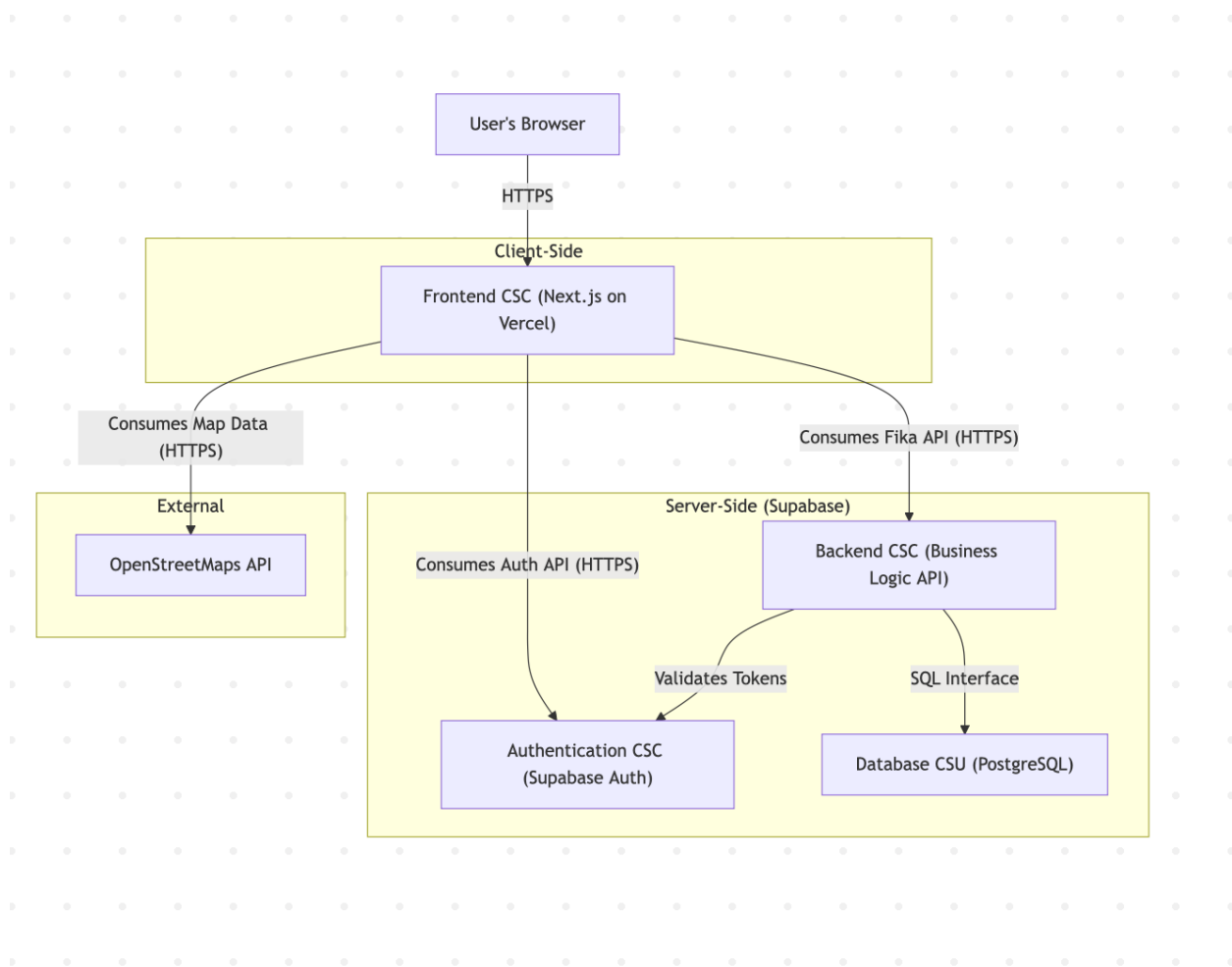
UML Use Case Diagram:



Deployment Diagram:



Component Diagram:



6.3 Detailed CSC and CSU Descriptions Section

This section details the Computer Software Components and Computer Software Units that comprise the fika application. The system is divided into logical components based on the three-tier architecture (Frontend, Backend, Database).

The Frontend CSC is composed of React components and Next.js pages which serve as the CSUs. The Backend CSC consists of service modules that interface with Supabase. The Data CSC consists of the database schemas and types.

6.3.1 Class Descriptions

The following sections provide the details of key classes (React components and Service modules) used in the fika application. These classes are selected to represent the core functionality of the discovery and logging systems.

6.3.1.1 Detailed Class Description: CafeMap

The CafeMap class is responsible for rendering a static location map for a specific cafe within the Cafe Details Page. Due to the Next.js server-side rendering environment, this component utilizes dynamic imports to load the Leaflet library only on the client side.

- Purpose: To convert a text-based address into geographical coordinates using the Nominatim API and display a pinned marker on an interactive map.
- Fields (Props & State):
 - address: String (Prop) - The physical address of the cafe to be geocoded.
 - cafeName: String (Prop) - The name of the cafe used for the marker popup.
 - coordinates: Object { lat, lon } (State) - The geocoded latitude and longitude derived from the address.
 - loading: Boolean (State) - Tracks the status of the asynchronous geocoding request.
 - isClient: Boolean (State) - A flag used to ensure Leaflet components only render after the component has mounted on the client.
- Methods:
 - useEffect(setupMarkerIcon): Fixes Leaflet's default icon path issues by manually overriding the icon URLs for the production environment.
 - geocodeAddress(): An asynchronous function triggered when the address prop changes. It fetches data from the OpenStreetMap Nominatim API (nominatim.openstreetmap.org) to convert the string address into coordinates.
 - MapContainer Render: Dynamically renders the map with a TileLayer and Marker only if coordinates are successfully resolved.

6.3.1.2 Detailed Class Description: CafeService

The CafeService is a utility class residing in the application layer. It acts as the abstraction layer between the Frontend UI and the Supabase client, handling data fetching and filtering logic.

- Purpose: To construct and execute database queries against the Supabase PostgreSQL instance, ensuring that raw database logic is decoupled from the UI components.
- Fields:
 - supabaseClient: The authenticated instance of the Supabase client used to make requests.
- Methods:

- `fetchAllCafes()`: Returns a list of all cafes stored in the database.
- `fetchCafeById(id)`: Accepts a UUID and returns the detailed metadata for a single cafe.
- `searchCafes(filters)`: Accepts a `FilterObject` (containing boolean flags for wifi, parking, etc.) and constructs a dynamic SQL query to return matching records.
- `getAggregateRating(cafeId)`: Queries the Review table to calculate the average score for a specific cafe.

6.3.1.3 Detailed Class Description: `SuggestCafeForm`

The `SuggestCafeForm` class manages the interface for users to propose new cafes for the platform. It utilizes a modal dialog and integrates with Server Actions to handle data submission.

- **Purpose:** To collect structured data about a potential new cafe, including categorical attributes and boolean features, and submit this to the backend.
- **Fields:**
 - `isOpen`: Boolean (Prop) - Controls the visibility of the modal dialog.
 - `state`: Object (Hook) - derived from `useActionState`, tracks the result message of the form submission.
 - `pending`: Boolean (Hook) - derived from `useFormStatus`, tracks if a submission is currently in progress to disable the submit button.
- **Methods:**
 - `SuggestCafeForm Render`: Renders a form containing text inputs (Name, Address) and Dropdown Selects populated by `Constants.public.Enums` (City, Seating, Parking, Vibe, Pricing, Busyness).
 - `formAction`: Binds the form submission event to the `suggestCafe` server action.
 - `SubmitButton`: A sub-component that monitors the pending status to provide visual feedback ("Submitting...") during network requests.

6.3.3 Detailed Data Structure Descriptions

This section details specific data structures used for storage and complex processing within the CSUs.

- **GeoJSON Feature Collection:** The `MapView` module utilizes the standard GeoJSON data structure to render cafe locations. This format is required for compatibility with the OpenStreetMaps/Leaflet libraries.
 - **Structure:** A JSON object containing a `"type": "FeatureCollection"` and an array

of "features." Each feature contains "geometry" (coordinates) and "properties" (cafe metadata like name and ID).

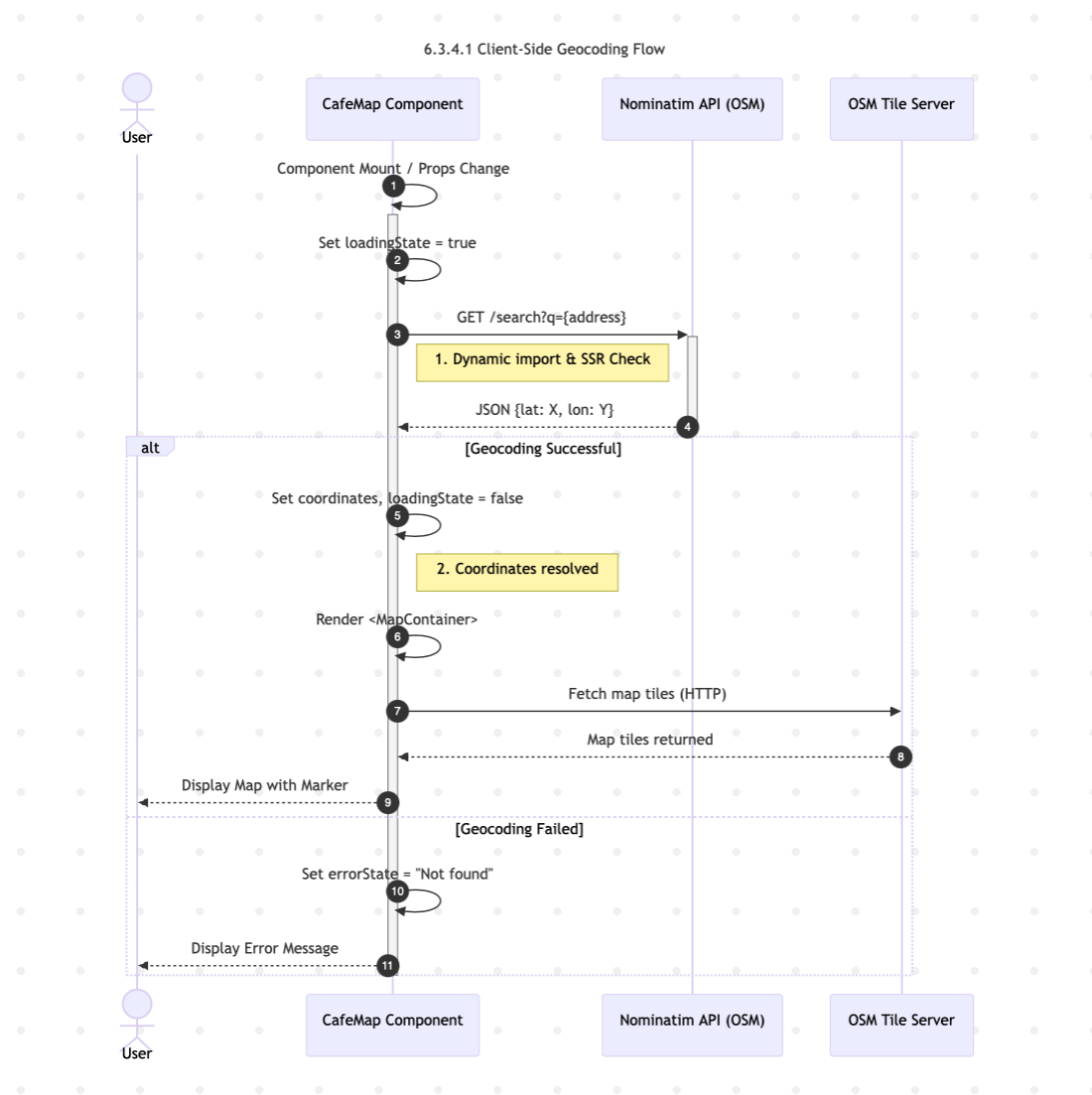
- JSONB Amenities Blob: To allow for flexible filtering without altering the database schema frequently, cafe amenities are stored in a PostgreSQL JSONB data structure within the CafeTable.
 - Structure: {"wifi": true, "parking_lot": false, "street_parking": true, "seating_capacity": "medium"}.
 - Purpose: This allows the CafeService to perform efficient key-value queries (e.g., amenities -> 'wifi' = 'true') directly within the database engine.

6.3.4 Detailed Design Diagrams

This section provides visual representations of the dynamic behavior and static structure of the fika system. These diagrams bridge the gap between the code specifications and the architectural overview.

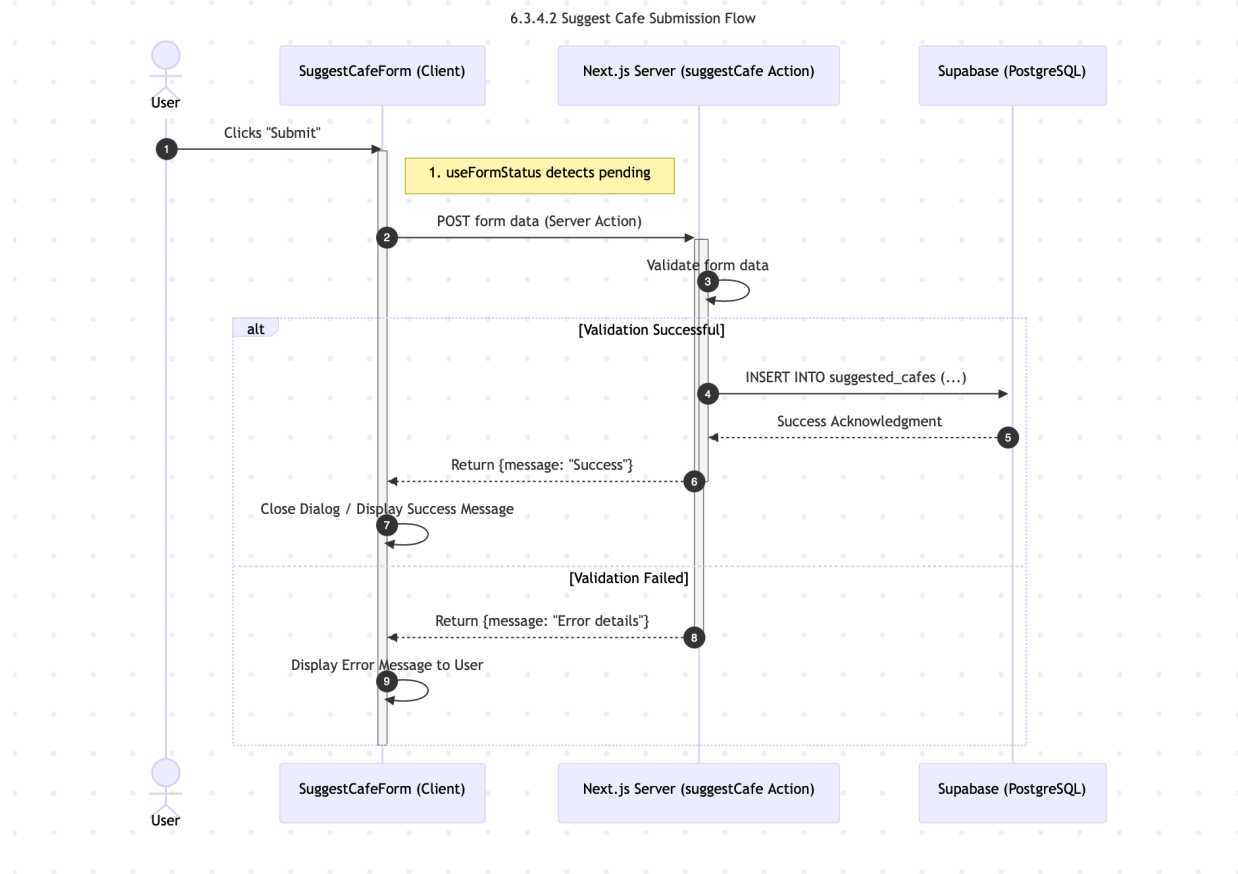
6.3.4.1 Sequence Diagram: Client-Side Geocoding (CafeMap)

This diagram details the flow of control when the CafeMap component loads the location based on the provided address.



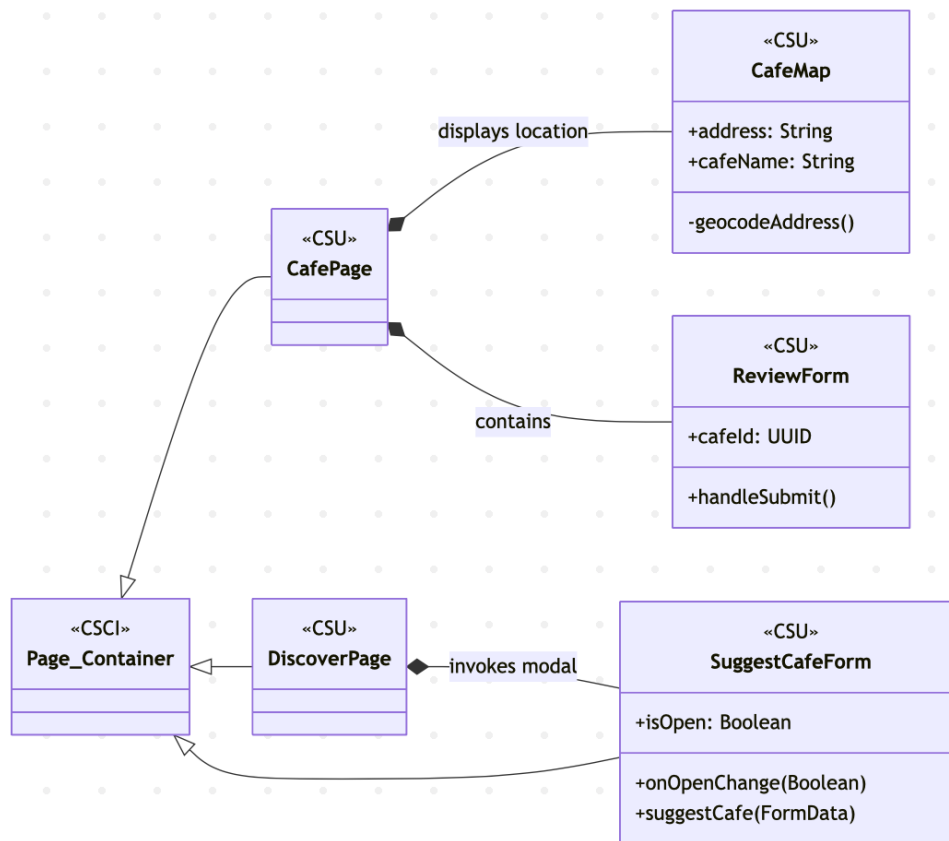
6.3.4.2 Sequence Diagram: Suggest Cafe Submission (Server Actions)

This diagram visualizes the communication flow for the SuggestCafeForm, utilizing the Next.js Server Action pattern (suggestCafe).



6.3.4.3 Class Diagram: Frontend Component Structure

This diagram shows the structural relationships (composition) between the key frontend units (CSUs). Member visibility is Public (+).



6.4 Database Design and Description

The fika application utilizes a relational database implemented in PostgreSQL, hosted and managed via Supabase. The database is designed to ensure data integrity between Users, Cafes, and Reviews while allowing for flexible querying of cafe amenities.

6.4.1 Database Design ER Diagram

The Entity-Relationship diagram below depicts the schema for the fika database. It illustrates the relationships between the core entities: profiles (Users), cafes, reviews, and the join table for saved_cafes (Favorites).

6.4.2 Database Access

Database access is managed through the Supabase Client library, which interacts with the PostgreSQL database via the PostGREST API.

- **Connection:** The application connects using a singleton instance of the SupabaseClient initialized with the Project URL and the Public Anon Key (for public reads) or the Service Role Key (for administrative tasks, used only in secure server-side contexts).
- **Query Method:** All queries are constructed using chained methods (e.g., `.from('cafes').select('*').eq('wifi', true)`). This abstraction prevents SQL injection attacks by sanitizing inputs before they reach the database layer.
- **Latency Management:** To ensure performance requirements (search results < 3 seconds), the cafes table is indexed on geographical coordinates and the JSONB amenities column.

6.4.3 Database Security

Security is implemented using PostgreSQL's Row Level Security (RLS) policies, managed through the Supabase dashboard.

- **Public Read Access:** The cafes and reviews tables have a "Enable Read Access for All" policy, allowing unauthenticated users to view content.
- **Authenticated Write Access:** The reviews and saved_cafes tables have strict policies (e.g., `auth.uid() = user_id`). This ensures that a user can only create, edit, or delete their own reviews and can never modify data belonging to another user.
- **Environment Variables:** All database connection keys are stored in environment variables (`.env.local`), ensuring that sensitive credentials are never committed to the version control system.